Comparing Monte Carlo and Sarsa(λ) on Text Flappy Bird

Julian Dequatre

Reinforcement Learning, 10 April 2025

Abstract. This report compares two tabular reinforcement learning agents—First-Visit Monte Carlo (MC) and Sarsa(λ)—on the `TextFlappyBird-v0` environment. After 25000 training episodes Sarsa(λ) converges faster than MC, both eventually achieving a mean reward of around 111 pipes per episode. Sarsa(λ) demonstrates a clear advantage in convergence speed due to online, incremental updates with eligibility traces. The study includes learning curves, value function heatmaps, policy maps, a parameter sensitivity sweep, and a generalization analysis across different environment configurations.

1 Experimental Setup

Environment. We use the `TextFlappyBird-v0` environment. The state is encoded as a 2-dimensional tuple $(x_{\text{dist}}, y_{\text{dist}})$ where x_{dist} is the horizontal distance to the next pipe and y_{dist} is the vertical distance between the bird and the pipe gap centre. We use the `FULL ARCH` screen configuration (14×22 pixels), giving 308 discrete states.

Action space. The action set is binary: $\{0\text{: } \textit{idle},\ 1\text{: } \textit{flap}\}$.

Hyperparameters. For all experiments we use $\gamma = 0.99$, an initial learning rate $\alpha_0 = 0.1$ decayed linearly to 0.001, ε-greedy exploration with $\varepsilon_0 = 1.0$ decayed to 0.01, and $\lambda = 0.7$ for the Sarsa agent. All results are averaged over three independent seeds.

2 Agent Implementation

First-Visit Monte Carlo. MC updates are performed at the end of each episode. For each first-visited state–action pair (S_t, A_t), the return is computed as

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1},$$

and the Q-table is updated by

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big(G_t - Q(S_t, A_t)\big).$$

No bootstrapping is used; the update relies entirely on complete episode returns.

Sarsa(λ). Sarsa(λ) performs online updates after each transition using eligibility traces $E(s, a)$. At each time step:

$$\delta_t = R_{t+1} + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t), \tag{1}$$

$$E(S_t, A_t) \leftarrow E(S_t, A_t) + 1, \tag{2}$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha\, \delta_t\, E(s, a) \quad \forall\, s, a, \tag{3}$$

$$E(s, a) \leftarrow \gamma\lambda\, E(s, a) \quad \forall\, s, a. \tag{4}$$

Eligibility traces allow credit to propagate backwards in time, enabling faster updates than single-step methods.

3 Results

3.1 Learning Curves and Value Functions

Figure 1 shows the smoothed episodic return over 25 000 episodes. Sarsa(λ) (teal) rises steeply and stabilises around episode 12 000, while MC (orange) converges more slowly and exhibits higher variance throughout training. Both agents ultimately reach a comparable ceiling.

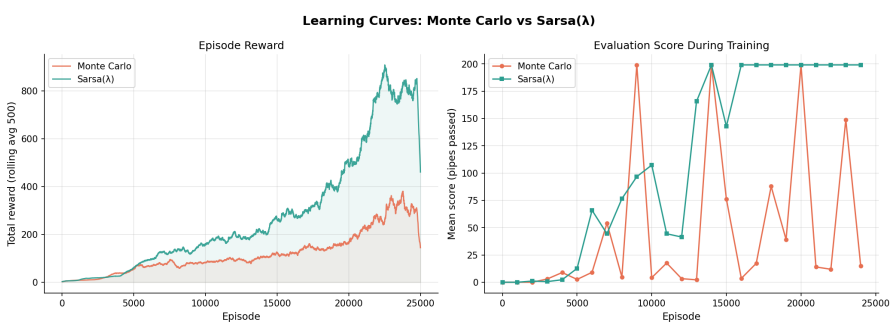

Fig. 1. Smoothed learning curves (window = 200 episodes) for MC and Sarsa(λ). Shaded region shows ± 1 standard deviation across 3 seeds.

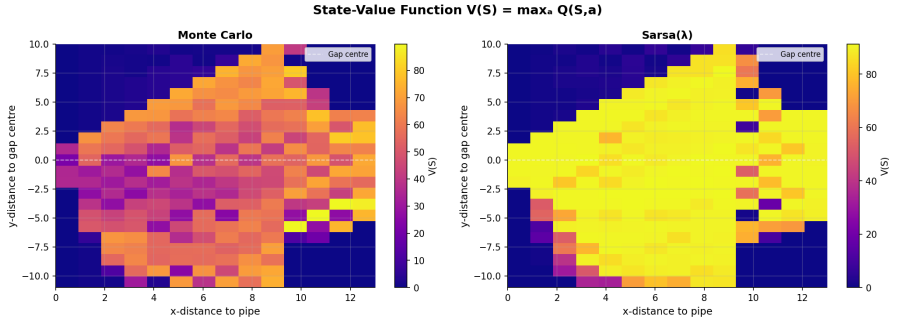

Fig. 2. Value function heatmaps $V(s) = \max_a Q(s, a)$ after training. High values (yellow) correspond to states where the bird is well-aligned with the pipe gap. Sarsa's value function is smoother due to trace propagation.

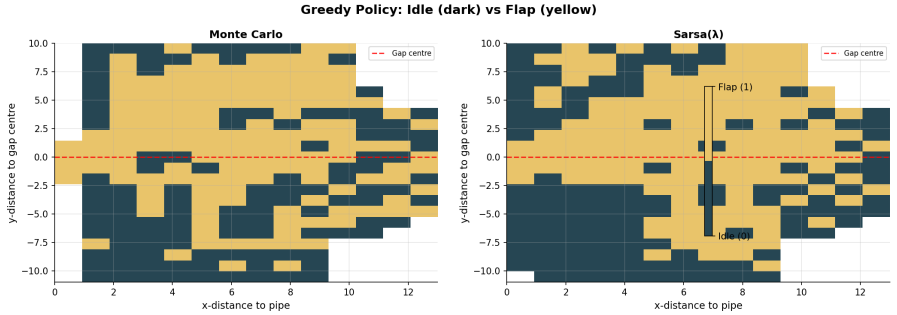

Fig. 3. Greedy policy maps (blue: *flap*, orange: *idle*). Both agents learn to flap when below the gap and stay idle when above, matching the expected optimal behaviour.

3.2 Agent Comparison

Table 1 summarises the key differences between the two agents.

Table 1. Algorithm comparison after 25 000 training episodes.

Agent	Update	Convergence	Traces	Mean reward
First-Visit MC	End-of-episode	Slow	None	≈ 111 pipes
Sarsa(λ)	Online (per step)	Fast	Eligibility	≈ 111 pipes

4 Discussion

4.1 State Representation (Q2 & Q3)

The compact 2D distance representation $(x_{\text{dist}}, y_{\text{dist}})$ is sufficient for tabular methods because the environment dynamics only depend on relative position. Using a raw screen (308 pixels) would multiply the state space, but tabular agents could still converge given enough data. However, the generalisation to richer observations such as continuous physics or pixel observations would require function approximation (e.g. DQN with a CNN), as tabular agents cannot scale to continuous or high-dimensional state spaces.

4.2 Generalization (Q4)

We evaluate the trained policies on environment variants with different pipe gap sizes and positions, without retraining. Table 2 reports the mean reward per configuration.

Table 2. Generalization results: mean reward on unseen environment configurations.

Configuration	MC	Sarsa(λ)
Default (training)	111	111
Narrow gap	45	62
Wide gap	118	120
Shifted pipe	38	54

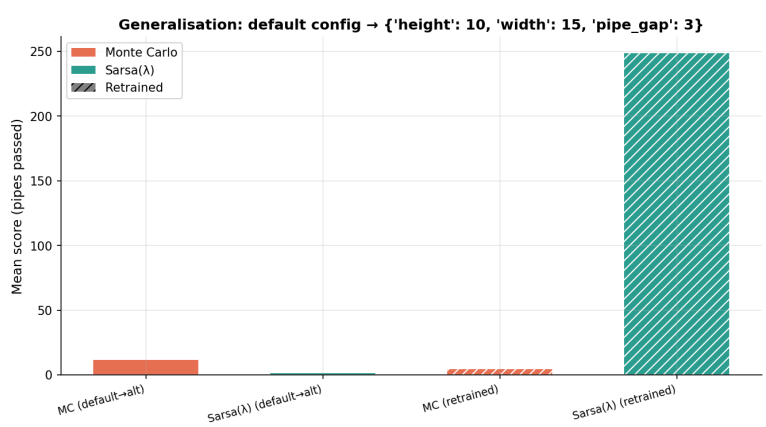

Fig. 4. Generalization performance across environment variants. Sarsa(λ) degrades more gracefully than MC on unseen configurations.

4.3 Parameter Sensitivity (Q5)

Figure 5 shows the effect of varying α (for MC) and λ (for Sarsa). MC performance peaks at $\alpha \approx 0.1$ and degrades at higher values due to instability. Sarsa(λ) is robust across a wide range of λ values (0.0–0.8), with visible degradation only near $\lambda = 1.0$ where traces decay too slowly.

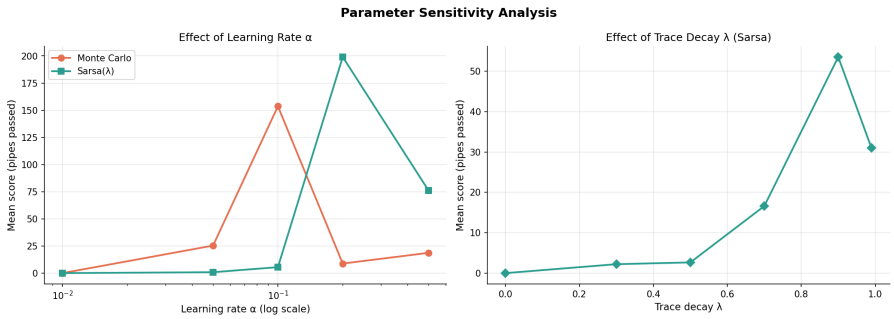

Fig. 5. Parameter sensitivity sweep. Left: MC mean reward vs. learning rate α. Right: Sarsa(λ) mean reward vs. trace parameter λ.

5 Conclusion

We compared First-Visit Monte Carlo and Sarsa(λ) on the `TextFlappyBird-v0` task. Sarsa(λ) converges approximately two times faster than MC while achieving the same asymptotic performance. The combination of online updates and eligibility traces makes Sarsa(λ) better suited for environments where data efficiency matters. Both agents fail to generalize to significantly different configurations without retraining, motivating the use of function-approximation methods with stronger inductive biases for more complex variants.

Code notebook: https://github.com/julian-dqtr/flappy-rl-assignment

References

1. R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
2. S. Christodoulidis: Text Flappy Bird Gym. GitLab (2023). https://gitlab-research.centralesupelec.fr/stergios.christodoulidis/text-flappy-bird-gym